Session 18 — Study Guide: Indexes, Transactions, and Closing the Relational Picture

Where the session ends

This session closes Block 4 (SQL/PostgreSQL). You now have a real, seven-table schema (the six from Session 17, plus `activity_logs`), a real index proven to speed up a real query, and a real, atomic transaction pattern proven two ways (commits, rolls back).

The real starting point: a short, bounded verification, not a re-teach

Class opens with a fresh, shared reseed and a quick check of Session 17's own homework (`queries.sql` — LEFT JOIN + aggregate functions + GROUP BY). If you finished it, you show it briefly. If not, you get real working time, with the instructor circulating — this is not a re-teach, so pay attention even if you're in recovery, since new material starts right after.

Why duplicated data is a real problem (not just untidy)

Imagine `projects` stored a workspace's own name directly, instead of looking it up through `workspace_id`. Renaming a workspace would then require updating every single project row that belongs to it — miss one, and the database now holds two different real answers to "what is this workspace called?" Our real schema avoids this: one real fact, one real place it lives, reached through a foreign key. This reasoning (not any named formal theory) is what "normalization" protects against.

CREATE INDEX — proven, not assumed

An index is a real, separate structure Postgres can use instead of scanning every row. It only matters at real volume — with four rows, scanning all of them is already the cheapest plan. We bulk-seeded 5,000 real rows, proved a real `Seq Scan` (cost 176.55), built `CREATE INDEX idx_issues_status ON issues(status);`, and proved a real `Index Scan` (cost 8.45) on the identical query. An index isn't free: every future write to that column does a little more work, and it costs real disk space — a deliberate trade, never a default applied everywhere.

Transactions — all or nothing, proven both ways

`BEGIN` / … / `COMMIT` makes every statement in between permanent together, or (with `ROLLBACK`) undoes all of them together. The real example: generating a project's next issue key (`WEB-3`) needs its own counter (`projects.issue_seq`) read and incremented atomically, so two real concurrent requests can never generate the identical key:

TEXT

```
BEGIN;
UPDATE projects SET issue_seq = issue_seq + 1 WHERE id = 1 RETURNING issue_seq;
INSERT INTO issues (project_id, key, title, status) VALUES (1, 'WEB-3', 'New issue',
'todo');
COMMIT;
```

We proved `COMMIT` keeps it, and separately proved `ROLLBACK` leaves `issue_seq` and the new issue completely untouched — nothing partial ever persists.

Teamwork: a new table, a new index shape, and an explicit lock (self-learned)

Your team built `activity_logs` (reusing the ON DELETE categories you already know), researched *compound* (multicolumn) indexes, and researched `SELECT ... FOR UPDATE` — an explicit row lock, the alternative way to do the identical atomic issue-key job. You proved, with two real terminal windows at once, that the lock genuinely blocks a second transaction until the first commits.

Homework: apply it yourself

Choose and build your own index for a real question you identify. Combine a transaction with a constraint you already know (`UNIQUE`) across two tables, atomically. Reflect, in writing, on what a schema migration tool would have automated across this whole block — without being shown one yet.

Common beginner mistakes

- Expecting an index to matter on a tiny table — it needs real volume to matter at all.
- Thinking `UPDATE 1` printing means the change is already permanent — only `COMMIT` makes it so.
- Confusing `issue_seq` with the table's own `id` — one is a per-project counter, the other is the row's own internal primary key.
- Running `SELECT ... FOR UPDATE` with no open `BEGIN` and wondering why nothing blocks.

A preview of what's still missing

`sql-practice/` has been a real, parallel practice project this whole block — `api/` (the real backend) never changed. The next block moves this same real relational thinking into the real app, using a tool that manages schema changes and queries for you.

Not yet

Relational-design theory as named, formal rules; named transaction isolation levels as formal theory; window functions; any object-relational mapping tool or schema-migration workflow, by name or syntax; any frontend framework.

Glossary

- **Index** — a real, separate structure that lets the database find matching rows without scanning every one.
- **Compound (multicolumn) index** — an index built on more than one column, serving a filter on the first and a sort on a later one.
- **Transaction** — a group of statements that become permanent together (`COMMIT`) or are undone together (`ROLLBACK`).
- **`SELECT ... FOR UPDATE`** — reads a row and holds an explicit lock on it until the transaction ends, blocking any other transaction from also locking that row in the meantime.
- **Update anomaly** — the real inconsistency risk created when the same fact is stored in more than one place.

Review questions

1. Why does an index need real volume of data before it actually matters?
2. What's the real cost an index always carries, even when it speeds up reads?
3. What does `ROLLBACK` guarantee that simply "not running `COMMIT` yet" doesn't already give you?
4. What's the real difference between `UPDATE ... RETURNING` and `SELECT ... FOR UPDATE` for the identical atomic problem?

Answers

1. With few rows, a full scan is already the cheapest real plan — the contrast only shows up once there's enough real data for scanning everything to actually cost something.
2. Every future write to the indexed column must also update the index, and the index itself takes real disk space.
3. `ROLLBACK` actively, explicitly undoes every statement since `BEGIN`, restoring the exact prior state — "not committing yet" alone still leaves the transaction open and locks held.
4. Both reach the identical atomic result for this narrow case. `SELECT ... FOR UPDATE` holds an explicit lock you can keep while running more logic before deciding what to write; `UPDATE ... RETURNING` is simpler when there's nothing to do in between.

What you should be able to explain now

Why duplicated data is a real risk, not an aesthetic complaint. When an index is worth its own real cost, and how to prove it changed the real query plan. What a transaction guarantees, proven both ways. What an explicit row lock actually holds, and when you'd reach for it over a simpler atomic statement.